Résumé de Cours Python — Syntaxe

Fonctions · Conteneurs · Modules · Fichiers · POO

1. Fonctions

1.1 Définition et appel

```
def nom_fonction(param1, param2):
# corps de la fonction
return valeur
```

```
# Appel normal
resultat = nom_fonction(val1, val2)

# Appel avec arguments nommés (ordre libre)
resultat = nom_fonction(param2=val2, param1=val1)
```

1.2 Nombre variable d'arguments

```
def add(a, b, *args): # *args → tuple de paramètres positionnels supplémentaires
s = a + b
for n in args:
s += n
return s

l = [2, 3, 4, 5]
add(*l) # 'dé-liste' l en arguments
```

```
def exemple(**kwargs): # **kwargs → dict de paramètres nommés supplémentaires
for k in kwargs:
print(k, '=', kwargs[k])

exemple(x=4, h='hache') # ou exemple(**{'x':4, 'h':'hache'})
```

1.3 Fonction Lambda (anonyme)

```
f = lambda x: x**2 # équivalent à def f(x): return x**2

# Appel immédiat
(lambda x, y: x + y)(3, 4) # → 7

# Avec plusieurs paramètres
g = lambda x, y: x * y
```

1.4 Portée des variables

```
x = 10 # variable globale

def ma_fonction():
y = 5 # variable locale
global x
x = 20 # modifie la variable globale
```

1.5 Gestion des exceptions

```
try:
resultat = 10 / 0
```

```
except ZeroDivisionError:
print('Division par zéro !')
except Exception as e:
print('Erreur :', e)
finally:
print('Toujours exécuté')
```

2. Conteneurs

2.1 Chaînes de caractères (str) — immuables

```
s = 'Bonjour' # guillemets simples
s = "Bonjour" # guillemets doubles
s = '''texte
multi-lignes''' # triple guillemets

# Concaténation & répétition
s1 + s2 # concaténation
s * 3 # répétition

# Accès & tranches (slicing)
s[0] # 1er caractère
s[-1] # dernier
s[1:4] # indices 1,2,3 (fin exclue)
s[::2] # un caractère sur deux
```

```
# Méthodes utiles
s.upper() / s.lower() # majuscules / minuscules
s.strip() # supprime espaces en début/fin
s.split(',') # découpe en liste
','.join(liste) # joint une liste en chaîne
s.replace('a','b') # remplace occurrences
s.find('mot') # index de 1ère occurrence (-1 si absent)
len(s) # longueur
```

2.2 Listes — mutables, ordonnées

```
lst = [1, 'texte', 3.14] # création
lst = list(range(5)) # [0,1,2,3,4]

lst[0] # accès
lst[1:3] # tranche
lst.append(x) # ajoute en fin
lst.insert(i, x) # insère à l'indice i
lst.remove(x) # supprime 1ère occurrence de x
lst.pop(i) # supprime et retourne l'élément i
lst.sort() # tri en place
lst.reverse() # inverse en place
len(lst) # longueur
x in lst # test d'appartenance
```

2.3 Tuples — immuables, ordonnés

```
t = (1, 2, 3) # création
t = 1, 2, 3 # parenthèses optionnelles
```

```
t = (5,) # tuple à 1 élément (virgule obligatoire)

t[0] # accès
a, b, c = t # déballage (unpacking)
```

2.4 Dictionnaires — mutables, clé→valeur

```
d = {'cle1': val1, 'cle2': val2}
d = dict(cle1=val1, cle2=val2)

d['cle1'] # accès
d['nouvelle'] = 99 # ajout / modification
del d['cle1'] # suppression
'cle1' in d # test clé

d.keys() # vue des clés
d.values() # vue des valeurs
d.items() # vue (clé, valeur)
d.get('cle', defaut) # accès sécurisé
d.update({'a': 1})
```

2.5 Ensembles (set) — non ordonnés, sans doublon

```
s = {1, 2, 3}
s = set([1, 2, 2, 3]) # → {1, 2, 3}

s.add(4) # ajoute un élément
s.remove(2) # supprime (erreur si absent)
s.discard(2) # supprime (sans erreur)

s1 | s2 # union
s1 & s2 # intersection
s1 - s2 # différence
```

3. Modules et Packages

3.1 Importation

```
import module # import complet
import module as alias # avec alias
from module import fonction # import sélectif
from module import fonction as f # sélectif + alias
from module import * # tout importer (déconseillé)
```

3.2 Créer un module

```
# Fichier : puissance.py
def carre(x):
return x ** 2

def cube(x):
return x ** 3

# Bloc exécuté uniquement si lancé comme script
if __name__ == '__main__':
```

```
print(carre(4))
```

3.3 Packages

```
# Structure
# package1/
# __init__.py (fichier vide ou d'init)
# module1.py

import package1.module1
package1.module1.fonction1()

# ou
from package1 import module1
module1.fonction1()
```

3.4 Modules courants

Module	Utilité principale
math	sin, cos, sqrt, pi, exp, log…
random	randint, random, choice, shuffle…
os	gestion fichiers/répertoires système
sys	paramètres environnement d'exécution
time	time(), sleep(), strftime()…
numpy	calcul scientifique, tableaux N-dim

4. Gestion des Fichiers

4.1 Ouverture / Fermeture

```
f = open('fichier.txt', mode) # ouvre le fichier
f.close() # ferme le fichier

# Modes principaux
# 'r' → lecture seule (fichier doit exister)
# 'w' → écriture (crée ou écrase)
# 'a' → ajout en fin (crée si inexistant)
# 'r+' → lecture + écriture
# 'rb' / 'wb' → mode binaire

# Bonne pratique : gestionnaire de contexte
with open('fichier.txt', 'r') as f:
contenu = f.read() # fichier fermé automatiquement
```

4.2 Écriture

```
with open('fichier.txt', 'w') as f:
f.write('une ligne\n') # écrit une chaîne
f.writelines(['a\n','b\n']) # écrit une liste de chaînes
```

4.3 Lecture

```
with open('fichier.txt', 'r') as f:
contenu = f.read() # tout le fichier (str)
ligne = f.readline() # une ligne
lignes = f.readlines() # liste de toutes les lignes

# Itérer ligne par ligne (efficace)
with open('fichier.txt', 'r') as f:
for ligne in f:
print(ligne.strip())
```

4.4 Navigation & Suppression

```
f.tell() # position courante (en octets)
f.seek(0) # retour au début
f.seek(n, 0) # depuis début ; 1=courant ; 2=fin

import os
os.remove('fichier.txt') # supprime le fichier
if os.path.exists('fich.txt'): # vérifie avant suppression
os.remove('fich.txt')
```

5. Programmation Orientée Objet (POO)

5.1 Définition d'une classe

```
class MaClasse: # Convention : PascalCase
def __init__(self, param1, param2): # Constructeur
self.attribut1 = param1 # attribut d'instance
self.attribut2 = param2

def ma_methode(self): # méthode
return self.attribut1
```

5.2 Instanciation et utilisation

```
obj = MaClasse(val1, val2) # création d'un objet (instance)
obj.attribut1 # accès à un attribut
obj.ma_methode() # appel d'une méthode
```

5.3 Encapsulation (attributs privés)

```
class Point:
def __init__(self, x, y):
self.__x = x # __ → attribut privé
self.__y = y

# Accesseur (getter)
def get_x(self):
return self.__x

# Mutateur (setter)
def set_x(self, x):
self.__x = x
```

5.4 Héritage simple

```
class Animal:
def __init__(self, nom):
self.nom = nom
def parler(self):
return '...'

class Chien(Animal): # Chien hérite de Animal
def parler(self): # redéfinition (override)
return 'Woof !'
def fetch(self): # méthode propre à Chien
return f'{self.nom} rapporte la balle'

d = Chien('Rex')
d.parler() # → 'Woof !'
d.fetch() # → 'Rex rapporte la balle'
```

5.5 Héritage multiple

```
class A:
def methode_a(self): ...

class B:
def methode_b(self): ...

class C(A, B): # hérite de A ET B
pass

c = C()
c.methode_a()
c.methode_b()
```

5.6 Méthodes spéciales utiles

Méthode	Déclenchée par…
`__init__(self, …)`	Création d'objet : `MaClasse(…)`
`__str__(self)`	`print(obj)` ou `str(obj)`
`__repr__(self)`	Représentation en console
`__len__(self)`	`len(obj)`
`__eq__(self, other)`	`obj1 == obj2`
`__lt__(self, other)`	`obj1 < obj2`